DG INTERNS HUB

CYBERSECURITY INTERNSHIP

WEEK 6

Web Security & VAPT Basics

Practical assessment using DVWA, Burp Suite and OWASP ZAP

Submitted by	**Akash Sanuria**
Intern ID	**DG/SEPTEMBER/CYBER/171**
Assignment	**Week 6**
Submission date	**14 October 2026**

What I did in Week 6

Introduction and lab scope

Aim

The aim of this week was to understand common web application vulnerabilities and get hands-on experience with basic vulnerability testing. I used an intentionally vulnerable application so that I could practice safely.

My lab

I used DVWA running locally at http://127.0.0.1:8080. I worked with Burp Suite Community Edition to look at web traffic and OWASP ZAP to run an automated scan.

Steps I followed

First I reviewed OWASP topics, then prepared the tools and DVWA. After that, I tested SQL Injection and both types of XSS, captured HTTP traffic in Burp Suite, and reviewed the scan alerts in ZAP.

Scope

All tests in this report were carried out in my own local DVWA training environment. The screenshots are from my practical work.

OWASP notes: SQL Injection

Task 1 - Basic concept

What it means

SQL Injection happens when a web application handles user input unsafely and that input becomes part of a database query. This may allow an attacker to change how the query works.

Simple example

Suppose a website searches for a user ID. If it directly adds the typed value to its SQL query, a specially crafted input may change the result. In DVWA, I used the SQL Injection exercise to observe this behavior.

Why it matters

SQL Injection can expose data, bypass checks or change database records. The exact impact depends on the application and database permissions.

How to prevent it

Developers should use parameterized queries, validate inputs on the server, give database accounts minimum permissions and avoid displaying detailed database errors.

OWASP category

SQL Injection is covered under Injection in the OWASP Top 10 (2021).

OWASP notes: XSS

Task 1 - Reflected and Stored XSS

What is XSS?

Cross-Site Scripting happens when a web application displays untrusted content without handling it safely. In some cases, a browser may run attacker-controlled JavaScript.

Reflected XSS

In reflected XSS, the submitted input comes back immediately in the page response. I tested this using the DVWA Reflected XSS exercise.

Stored XSS

In stored XSS, the input is saved by the application and may appear again when the page is opened. I tested this using the DVWA Stored XSS exercise.

Why it matters

XSS can affect users viewing the page, including changing page content or accessing information available to scripts.

Prevention

Use context-appropriate output encoding, safe DOM methods, input validation where suitable and a Content Security Policy as an additional protection.

OWASP notes: Other risks

Task 1 - Authentication and configuration

Broken Authentication

Authentication problems happen when login and session controls do not protect accounts properly. Examples include weak passwords, missing rate limits and unsafe session handling.

Prevention for authentication

Use multi-factor authentication where appropriate, secure session cookies, rate limiting and proper session expiry. Test login and logout behavior.

Security Misconfiguration

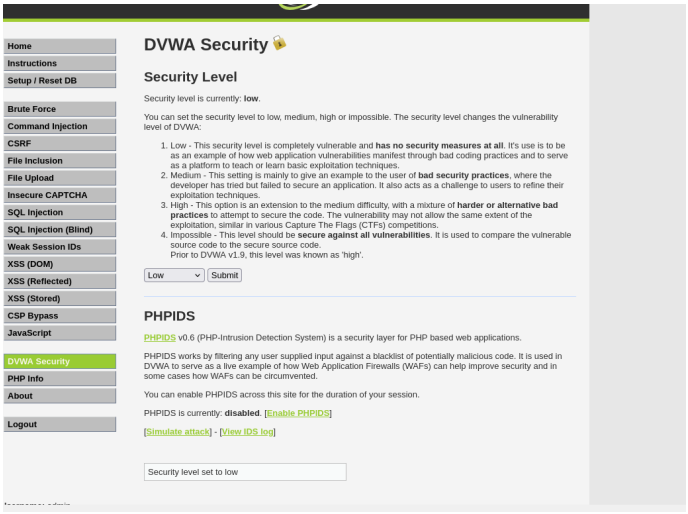
This means security settings are missing or unsafe. Examples include unnecessary services, default accounts, verbose error messages and missing browser security headers.

Prevention for configuration

Remove unused features, update software, set secure headers, turn off unnecessary debug information and review configuration regularly.

What I learned

A website can have security issues even when its pages appear to work normally. Reviewing configuration and session settings is important along with manual testing.



Setting up my lab

Task 2 - DVWA and security tools

DVWA

I ran Damn Vulnerable Web Application (DVWA) locally using Docker. The web application was accessible through my browser on port 8080.

Burp Suite Community

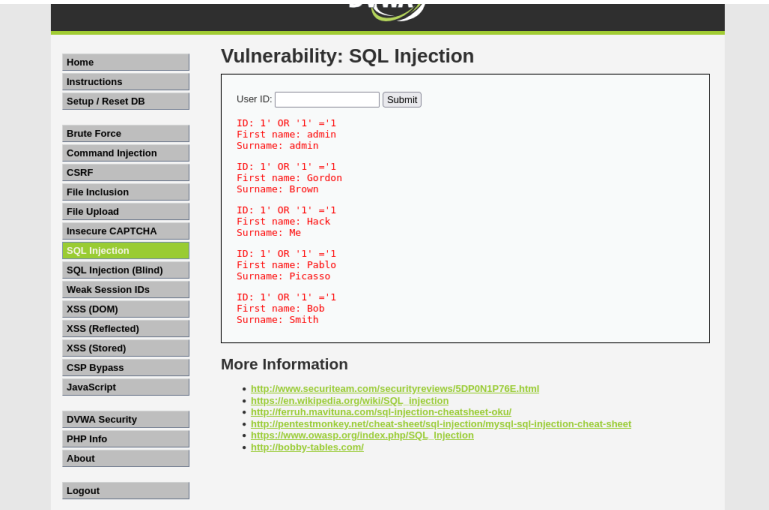
I used Burp Suite to view HTTP requests and responses while interacting with DVWA.

OWASP ZAP

I opened OWASP ZAP and used its automated scanning feature against my local DVWA target.

Evidence

The screenshots below show the working lab and tool setup.



Testing SQL Injection

Task 3 - Manual DVWA test

What I did

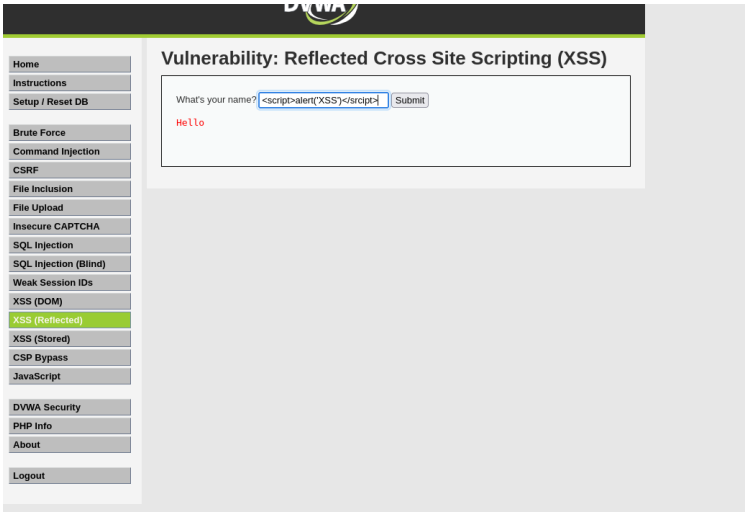
I opened the SQL Injection exercise in DVWA and entered test input in the user ID field. I compared the output with the normal response to understand how unsafe database queries can behave.

Observation

The exercise showed why input should be treated as data rather than part of a query. The screenshot below records the input and output from my test.

Fix

The main fix is to use prepared statements or parameterized queries, with proper server-side checks.



Testing Reflected XSS

Task 3 - Manual DVWA test

What I did

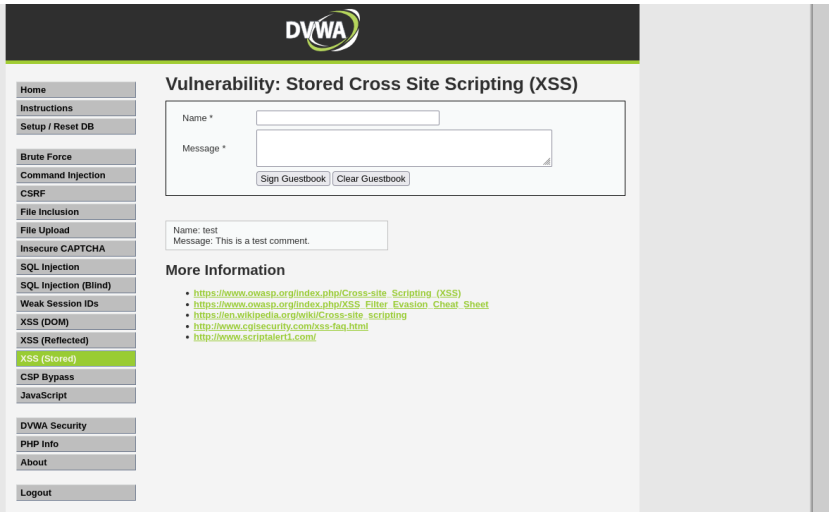
I opened the Reflected XSS exercise and submitted a test payload through the input field. I checked how the application displayed the submitted value.

Observation

This exercise helped me see the difference between showing text safely and allowing untrusted input to affect a browser page.

Fix

Use proper output encoding and avoid placing untrusted input directly into HTML or JavaScript contexts.



Testing Stored XSS

Task 3 - Manual DVWA test

What I did

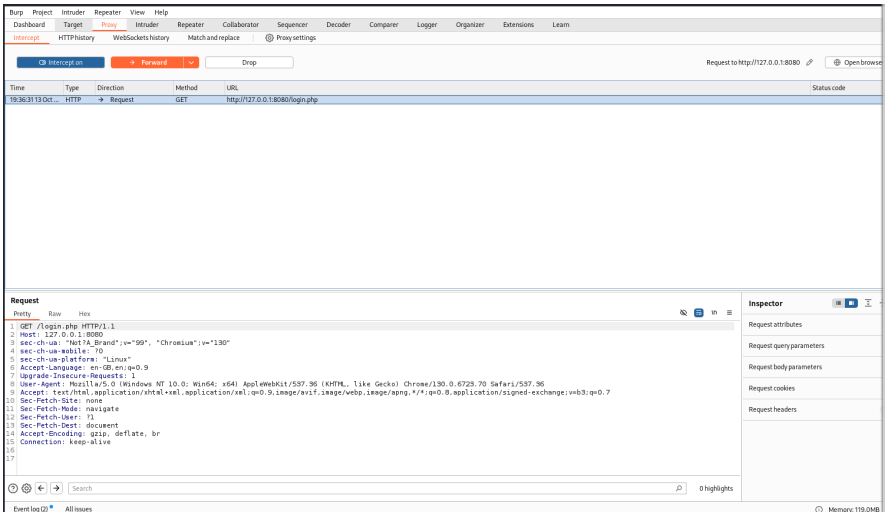
I used the Stored XSS exercise in DVWA and submitted a test value to a field that saves content. I then checked the page output.

Observation

Unlike reflected XSS, stored content may be shown again later. This makes it important to protect all users who view that page.

Fix

Apply safe output encoding when stored content is displayed and use secure HTML handling.



Checking traffic in Burp Suite

Task 3 - Request and response

What I did

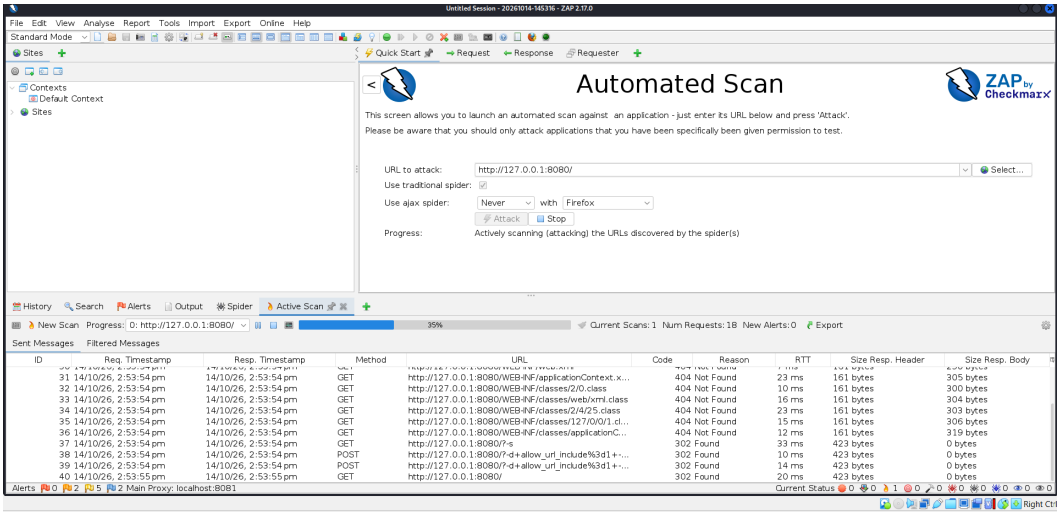
I opened DVWA through the browser configured for Burp Suite and checked the HTTP messages. I reviewed the request method, URL path, headers and the response status.

What I learned

Burp Suite helps show what the browser actually sends to the web server and what the server returns. It is useful for understanding how inputs are passed between a browser and an application.

Evidence

The screenshots show the captured HTTP request and response from the local lab.



Running the OWASP ZAP scan

Task 4 - Automated vulnerability scan

What I did

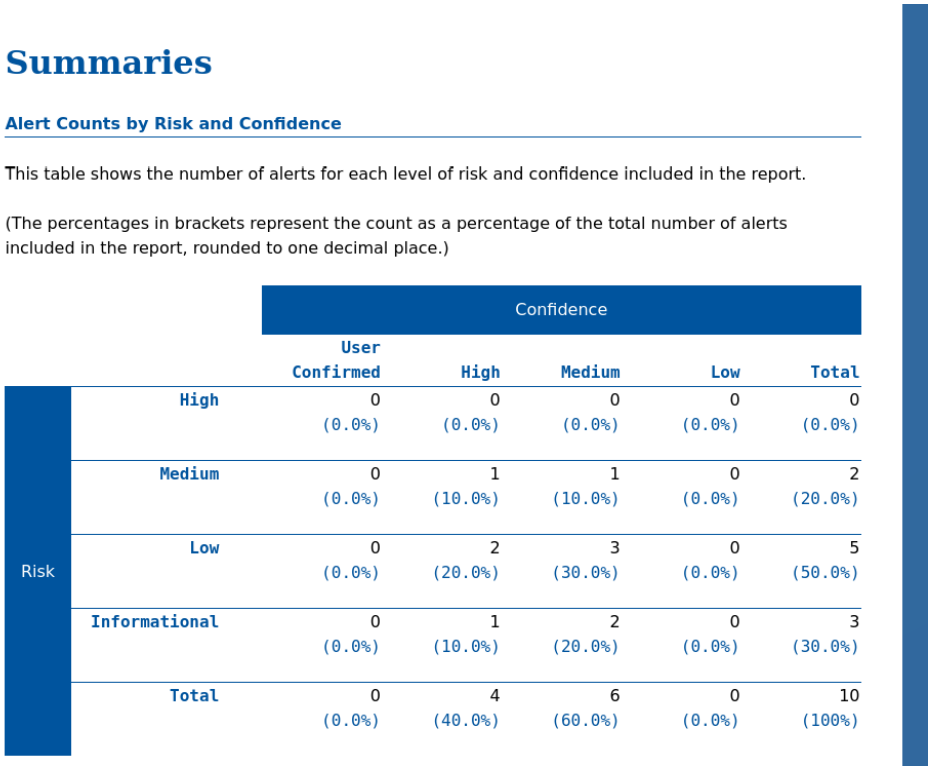
I entered the local DVWA address in OWASP ZAP and ran an automated scan. Once the scan finished, I reviewed the alerts and exported an HTML report.

What I found

The report grouped the findings by risk level. Some findings related to missing security headers and cookie settings.

Important point

An automated alert is a starting point for investigation. It should be checked before deciding the real-world impact.



Summaries

Alert Counts by Risk and Confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		User Confirmed	High	Medium	Low	Total
Risk	High	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
	Medium	0 (0.0%)	1 (10.0%)	1 (10.0%)	0 (0.0%)	2 (20.0%)
	Low	0 (0.0%)	2 (20.0%)	3 (30.0%)	0 (0.0%)	5 (50.0%)
	Informational	0 (0.0%)	1 (10.0%)	2 (20.0%)	0 (0.0%)	3 (30.0%)
	Total	0 (0.0%)	4 (40.0%)	6 (60.0%)	0 (0.0%)	10 (100%)

ZAP results and findings

Task 4 and 5 - Risk levels

Scan summary

The scan report listed 10 alert types: 0 High, 2 Medium, 5 Low and 3 Informational. These numbers describe alert categories, not necessarily individual affected pages.

Medium findings

Content Security Policy (CSP) Header Not Set; Missing Anti-clickjacking Header.

Low findings

Cookie No HttpOnly Flag; Cookie without SameSite Attribute; In Page Banner Information Leak; Server Leaks Version Information via Server response header; X-Content-Type-Options Header Missing.

Informational findings

Authentication Request Identified; Session Management Response Identified; User Agent Fuzzer.

My interpretation

The scan highlighted security configuration improvements. I would check each alert against the application before treating it as a confirmed exploitable weakness.

How the issues can be fixed

Task 5 - Mitigation methods

SQL Injection

Use prepared statements, server-side validation and restricted database permissions.

Reflected and Stored XSS

Encode output correctly, handle HTML safely and avoid unsafe DOM insertion. A CSP can provide additional protection.

Missing CSP and anti-clickjacking protection

Add a suitable Content Security Policy and use frame-ancestors or other appropriate framing protections.

Cookie flags

Use HttpOnly and SameSite settings where suitable; use Secure for cookies sent over HTTPS.

Information disclosure

Avoid unnecessary software version details and remove debugging information from production responses.

Missing X-Content-Type-Options

Set X-Content-Type-Options: nosniff where appropriate.

Retesting

After making changes, repeat the same manual checks and scan to confirm that the issue has been addressed.

What I learned

Conclusion and limitations

Conclusion

During Week 6, I practiced basic web security testing using DVWA, Burp Suite and OWASP ZAP. I learned how SQL Injection and XSS can happen, how to inspect HTTP requests, and how an automated scan presents possible security issues. The practical work made the OWASP topics easier for me to understand.

Limitations

This was a beginner-level exercise in a deliberately vulnerable local application. I did not assess a real production website. ZAP findings are automated alerts and need manual verification. The work does not represent a complete penetration test.

Next step

I want to improve my ability to validate findings, understand their impact and write clearer remediation notes.